



KEMENTERIAN
KETENAGAKERJAAN
REPUBLIK INDONESIA

PRESENSI



2025

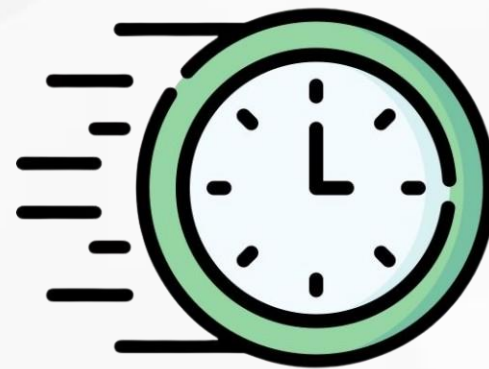


Smart **Creative Skills** Team ©

TATA TERTIB KELAS



Handphone
Harap Disimpan



Hadir Tepat Waktu



Dilarang
makan & minum



Berpakaian
Rapi



Angkat Tangan
Sebelum Bertanya



Izin Jika Ada
Keperluan

POSTURES AT WORK



designed by  freepik.com

SAFETY INDUCTION

- ☐ Posisi badan **rileks**.
- ☐ Jarak pandangan **mata** sekitar **45 cm**.
- ☐ **Duduk tegak** dengan sudut 90-100 derajat.
- ☐ **Jangan berdiam** selama berjam-jam

APERSEPSI

HW 8 Part B 17

- 2) a) spiral. Doesn't have bars, but does have spiral arms
- 2) b) barred spiral, has bars and spiral arms
- 2) c) SO, perfectly circular and has the start of spiral arms
- 2) d) irregular, no spiral arms, not circular, under no classification except irregu
- 2) e) spiral, has many spiral arms, but no bars
- 2) f) barred spiral, barred from bulge with spirals coming off
- 2) g) Elliptical, No spiral arms and almost a perfect circle
- 2) h) Cold dark matter, slow moving heavy exotic particles
- 13) It started from one point because it's expanding now
- 04)

Overthinking will
destroy your
happiness and your
mood. It'll make
everything worse
than it actually
is. Take a deep
breathe, exhale
and have faith.
What's meant to be
will be.



KEMENTERIAN
KETENAGAKERJAAN
REPUBLIK INDONESIA



Project Based
Learning



J.620100.016.01

MENULIS KODE DENGAN PRINSIP SESUAI GUIDELINES DAN BEST PRACTICES

Program Pelatihan :

Pengembangan Web dengan Node.js dan React

Smart **Creative Skills** Team ©

ROAD MAP PELATIHAN

01

TNA

02

Proglat :

Pengembangan
Web dengan Node.js
dan React

03

Judul Unit :

Menulis Kode Dengan Prinsip
Sesuai Guidelines Dan Best
Practices

07

08

09

10

TUJUAN

Setelah mengikuti pelatihan ini peserta kompeten menerapkan Menulis Kode Dengan Prinsip Sesuai Guidelines Dan Best Practices pada bahasa pemrograman Javascript sesuai dengan prosedur

MANFAAT

Mampu menuliskan Kode Dengan Prinsip Sesuai Guidelines Dan Best Practices pada bahasa pemrograman Javascript



OUTLINE



Literatur 2

Prinsip Penulisan Kode yang Baik



Literatur 4

Efisiensi & Optimasi Kode



Literatur 1

Pengenalan Coding Guidelines



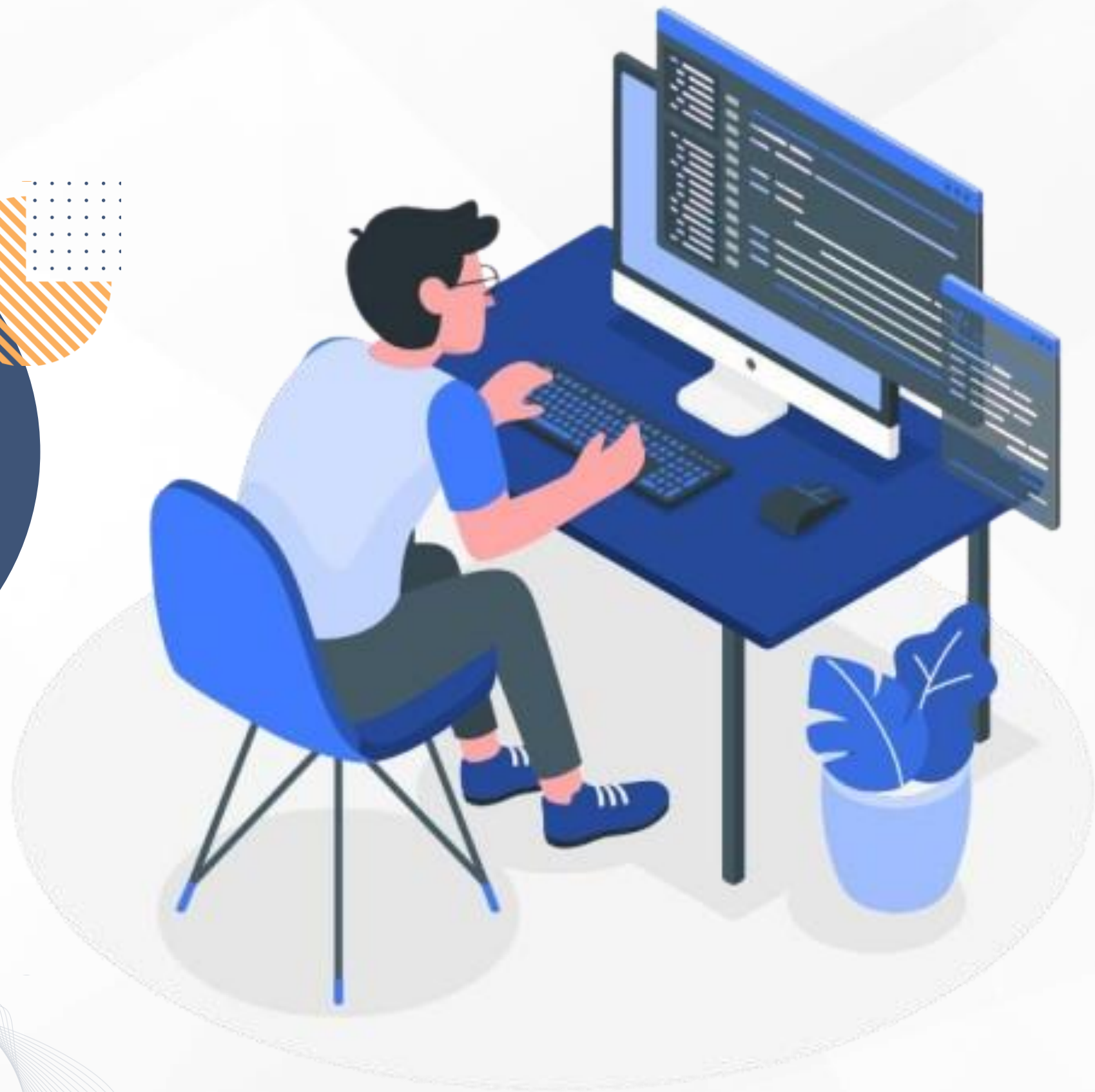
Literatur 3

Paradigma Pemrograman



Literatur 5

Penanganan Kesalahan (Error Handling)



01

Pengenalan Coding Guidelines



Apa itu Coding Guidelines?



Pedoman penulisan kode yang disepakati bersama agar seluruh tim memiliki gaya kode yang konsisten dan mudah dipahami.

Manfaat Coding Guidelines



- ✓ Kode lebih mudah dibaca
- ✓ Meminimalkan bug
- ✓ Memudahkan kolaborasi antar programmer

02

Prinsip Penulisan Kode yang Baik



Penamaan (Naming Convention)

Elemen	Konvensi	Contoh
Variabel	<code>camelCase</code>	<code>totalHarga</code>
Fungsi	<code>camelCase</code>	<code>hitungDiskon()</code>
Class	<code>PascalCase</code>	<code>UserProfile</code>
Konstanta	<code>UPPER_CASE</code>	<code>MAX_RETRY</code>

Komentar Efektif



```
// ✗ Tidak berguna  
// Menambahkan dua angka  
function add(a, b) {  
    return a + b;  
}
```

```
// ✓ Menjelaskan "mengapa"  
function calculateTax(amount) {  
    // Pajak 10% sesuai regulasi 2024  
    return amount * 0.1;  
}
```

Struktur dan Indentasi

```
1 // ✗ SALAH
2 ✓ function cekLogin(){
3 ✓ if(user){
4   console.log("Login berhasil");
5 }
6 }
7
```

```
1 // ✓ Benar
2 ✓ function cekLogin() {
3 ✓   if (user) {
4     console.log("Login berhasil");
5   }
6 }
7 |
```

03

Paradigma Pemrograman



Functional Programming

Kita sering mendengar berbagai paradigma pemrograman seperti object-oriented, procedural, dan functional. JavaScript adalah bahasa pemrograman multi-paradigma yang memungkinkan kita untuk secara bebas mencampur dan mencocokkan paradigma pemrograman.

Apa itu functional programming? yaitu sebuah paradigma dalam dunia programming yang menggunakan pendekatan fungsional untuk menyelesaikan masalah. Apakah jika kode program kita menggunakan function maka itu pasti disebut sebagai functional programming? ternyata tidak, paradigma ini mensyaratkan penggunaan pure function sebisa mungkin.

Keuntungan Functional Programming

Berikut beberapa keuntungan yang kita dapatkan ketika menggunakan paradigma functional programming

- Performa relatif lebih baik karena kode program menjadi relatif lebih singkat
- Kode program menjadi lebih terbaca
- Penggunaan pure function membuat kode program lebih mudah didebug

Metode-Metode Functional Programming



Berikut ini beberapa metode-metode dalam paradigma functional programming

- Recursive Function
- First-class Function
- Currying Function
- Declarative vs Imperative Pattern

Recursive Function

Recursive function maksudnya adalah function yang memanggil dirinya sendiri sampai suatu kondisi pemanggilan dirinya berhenti. Contoh:

```
// program untuk menghitung mundur hingga angka 1
function countdown(number) {
  // menampilkan number
  console.log(number);

  // mengurangi nilai number
  var newNumber = number - 1;

  // kondisi jika nomor masih diatas 0
  if (newNumber > 0) {
    countdown(newNumber);
  }
}

countdown(4);
```

First-class Function

First-class function maksudnya adalah menjadikan fungsi layaknya class atau dalam hal ini kita bisa mendeklarasikan fungsi di dalam fungsi yang lain.

Contoh:

```
function HitungLingkaran(radius) {  
  var pi = 22/7  
  
  function luas (r){return pi * r * r}  
  function keliling (r){return 2 * pi * r }  
  console.log("Luas:", luas(radius))  
  console.log("Keliling:", keliling(radius))  
}  
  
HitungLingkaran(7)
```

Currying Function

Fungsi currying adalah teknik untuk mengubah bentuk suatu fungsi dengan beberapa argumen menjadi sebuah fungsi anak yang berurutan. Fungsi currying akan mengembalikan fungsi baru yang sesuai dengan argumen berikutnya. Contoh:

```
function tambah (a, b){  
  return a + b  
}  
  
console.log(tambah(2, 5)) // 7  
  
// currying function  
function tambah (a){  
  return function(b){  
    return a + b  
  }  
}  
  
console.log(tambah(2)(5)) // 7
```

Currying Function

Apa Keunggulan Currying Function

pada contoh sebelumnya masih terasa Keunggulan dari currying function kecuali hanya untuk memberikan contoh bentuk dasarnya.

Teknik currying ini sangat berguna jika terdapat argumen dari fungsi itu yang membutuhkan komputasi berat

Currying Function

Contoh Komputasi berat tanpa currying function

```
// simulasi untuk komputasi berat
function getGreeting(){
  var x = 0
  for (i=0; i<1000000000; i++){
    x += i
  }
  return "Selamat pagi"
}

function say(callback, name) {
  var greeting = callback()
  console.log(greeting, name)
}

var users = ['Fuad', 'Hasan', 'Ismi', 'Budi', 'Azhar']

users.map(function(user) {
  return say(getGreeting, user)
}))
```

Currying Function

Contoh Komputasi berat dengan currying function

```
// simulasi untuk komputasi berat
function getGreeting(){
  var x = 0
  for (i=0; i<1000000000; i++){
    x += i
  }
  return "Selamat pagi"
}

function say(callback) {
  var greeting = callback()
  return function(name){
    console.log(greeting, name)
  }
}

var users = ['Fuad', 'Hasan', 'Ismi', 'Budi', 'Azhar']
var sayGreeting = say(getGreeting)

users.map(function(user) {
  return sayGreeting(user)
})
```

Declarative vs Imperative Pattern

Terdapat dua pattern atau pola ketika membuat kode program yaitu:

- Imperative berfokus pada gambaran bagaimana suatu program beroperasi, ada step by stepnya.
- Declarative berfokus pada apa yang harus dicapai oleh program tanpa menjabarkan step by stepnya.

Imperative Pattern

berikut ini contoh imperative pattern:

```
// Imperative: Bagaimana menghitung jumlah bilangan genap dalam array
function hitungBilanganGenap(arr) {
  var count = 0;
  for (var i = 0; i < arr.length; i++) {
    if (arr[i] % 2 === 0) {
      count++;
    }
  }
  return count;
}

var angka = [1, 2, 3, 4, 5, 6];
var jumlahGenap = hitungBilanganGenap(angka);
console.log(jumlahGenap); // Output: 3
```

Declarative Pattern

berikut ini contoh declarative pattern:

```
// Declarative: Apa yang ingin dicapai  
var angka = [1, 2, 3, 4, 5, 6];  
var jumlahGenap = angka.filter(function(x){return x % 2 === 0}).length;  
console.log(jumlahGenap); // Output: 3
```

04

Penanganan Kesalahan (Error Handling)



Validasi Manual

```
function hitungDiskon(harga, persen) {  
  if (typeof harga !== "number" || typeof persen !== "number") {  
    throw new Error("Input harus berupa angka");  
  }  
  return harga - (harga * persen) / 100;  
}
```

Try... Catch

```
function cekLogin() {  
  try {  
    if (user) {  
      console.log("Login berhasil");  
    } else {  
      throw new Error("User tidak ditemukan");  
    }  
  } catch (error) {  
    console.error("Terjadi kesalahan:", error.message);  
  }  
}
```

KESIMPULAN



Kesimpulan 1

Menulis kode dengan guideline
= kualitas + kolaborasi

Kesimpulan 2

Prinsip dasar meliputi penamaan,
struktur, komentar dan penanganan
kesalahan

APLIKASI

Buatlah sebuah fungsi yang berfungsi untuk melakukan pengecekan bilangan ganjil dan genap dengan menerapkan prinsip guidelines dan best practice dengan mengikuti instruksi yang diberikan pada Jobsheet.



EVALUASI

Kerjakan soal sesuai
dengan lembar evaluasi
yang telah dibagikan

Waktu Pengejaan
30 menit



TUGAS TAMBAHAN

Pelajari kembali tentang
Javascript Dasar





KEMENTERIAN
KETENAGAKERJAAN
REPUBLIK INDONESIA

SEKIAN DAN TERIMA KASIH

Sampai jumpa di pertemuan selanjutnya

Salam Kompeten !



2025



Smart **Creative Skills** Team ©